

Algorithmes de tri

1 Introduction

Le tri est un problème classique (et utile) de l'algorithmique utilisé sur un ensemble d'éléments possédant une relation d'ordre total (comme l'ordre des nombres et l'ordre lexicographique pour les caractères) afin de l'ordonner dans l'ordre croissant (ou décroissant).

En première année, trois algorithmes ont été étudiés : le tri par insertion, le tri par sélection et le tri à bulles, chacun de complexité quadratique dans le pire des cas. Cette section explore deux algorithmes plus efficaces : le tri par fusion (*mergesort*) et le tri rapide (*quicksort*).

Rappel

- *Tri en place* : un algorithme trie en place s'il ne nécessite qu'un espace mémoire supplémentaire négligeable (typiquement $O(1)$ ou $O(\log n)$), par opposition à un espace proportionnel à la taille du tableau.
- *Stabilité* : un algorithme de tri est stable si deux éléments égaux conservent leur ordre relatif après le tri.

2 Le paradigme diviser pour régner

Ce paradigme consiste à résoudre un problème en trois étapes :

- **Diviser** le problème initial en plusieurs sous-problèmes de même nature mais plus petits;
- **Régner** : résoudre chaque sous-problème récursivement (jusqu'à atteindre un cas de base suffisamment simple pour être résolu directement);
- **Combinaison** les solutions des sous-problèmes pour obtenir la solution du problème initial.

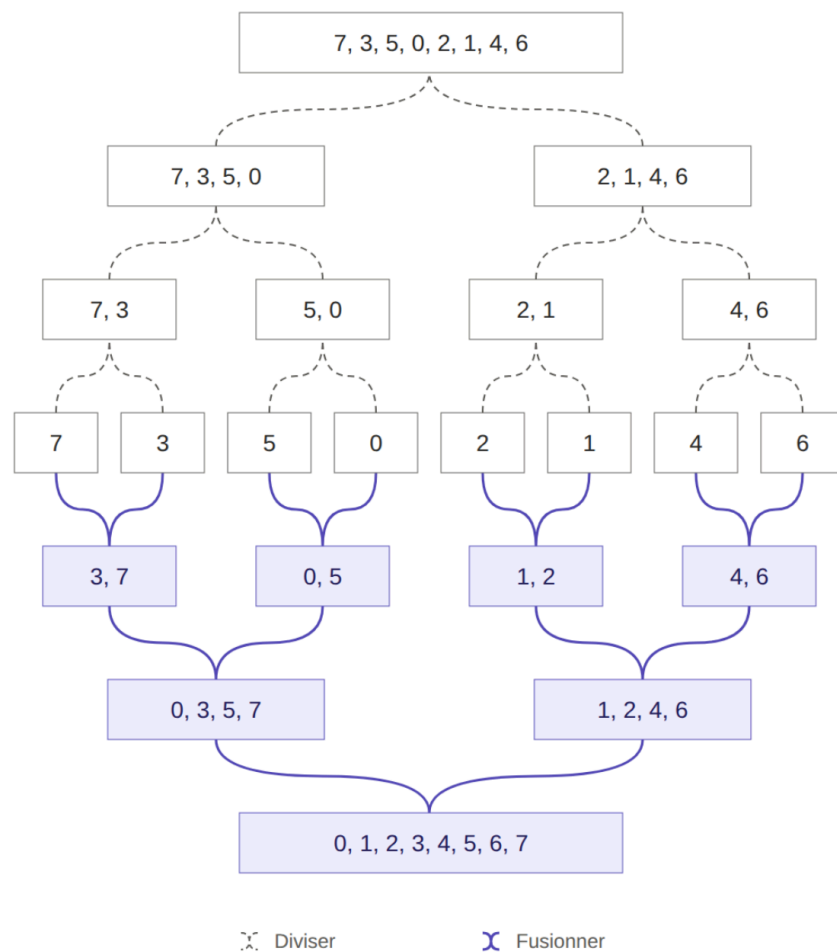
3 Le tri par fusion

3.1 Principe

C'est un tri, stable mais non en place, qui suit le paradigme diviser pour régner :

- **Diviser** : on divise le tableau en deux moitiés égales;
- **Régner** : on trie récursivement chacune des deux moitiés. La récursion s'arrête lorsqu'une moitié a une longueur de 0 ou 1;
- **Combinaison** : on fusionne les deux moitiés triées en un seul tableau trié, en comparant les éléments un à un.

3.2 Illustration



3.3 Implémentation

On commence par la fonction *fusion* qui prend deux sous-tableaux déjà triés et les combine en un seul tableau trié, en comparant seulement et à chaque fois les deux éléments en tête :

```
[ ]: def fusion(gauche, droite):  
    x = []  
    i1 = i2 = 0  
    while i1 < len(gauche) and i2 < len(droite):  
        if gauche[i1] <= droite[i2]:  
            x.append(gauche[i1])  
            i1 += 1  
        else:  
            x.append(droite[i2])  
            i2 += 1
```

```
x.extend(gauche[i1:])
x.extend(droite[i2:])
return x
```

En suite, la fonction récursive *tri_fusion* qui fait une descente en divisant le tableau en deux moitiés jusqu'à atteindre les cas de base; puis une remontée en combinant les moitiés triées par la fonction *fusion*.

```
[ ]: def tri_fusion(tab):
    if len(tab)<=1:
        return tab
    milieu = len(tab)//2
    gauche = tri_fusion(tab[:milieu]) # la descente
    droite = tri_fusion(tab[milieu:])
    return fusion(gauche, droite)     # la remontée
```

3.4 Complexité

Étape	Coût
Division	$\Theta(\log n)$ niveaux de récursion (on divise par 2 à chaque fois)
Fusion	$\Theta(n)$ à chaque niveau (chaque élément est examiné une fois par niveau)

Ce qui donne un total de $\Theta(n \log n)$

À faire : Confirmer cette complexité en utilisant le théorème maître.

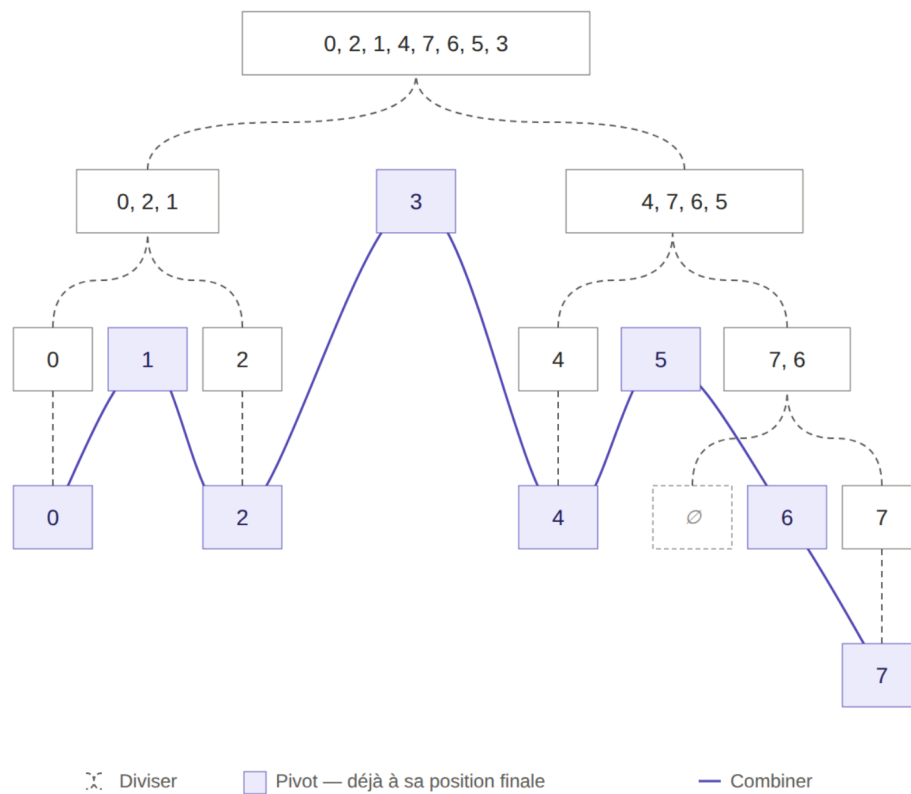
4 Le tri rapide

4.1 Principe

C'est un tri qui suit aussi le paradigme diviser pour régner, mais en déplaçant l'effort vers l'étape de division :

- **Diviser** : on choisit un élément du tableau appelé *pivot* et on divise le tableau autour de lui : Tous les éléments inférieurs au pivot sont placés avant lui et tous les éléments supérieurs après lui. À l'issue de cette étape, le pivot est à sa position définitive dans le tableau trié;
- **Régner** : on applique récursivement le même procédé sur les deux sous-tableaux gauche et droit. La récursion s'arrête lorsqu'un sous-tableau contient 0 ou 1 élément. Au fil de la récursion, chaque élément du tableau devient pivot exactement une fois, et c'est précisément à ce moment-là qu'il trouve sa position finale;
- **Combiner** : il n'y a ici (presque) rien à faire : on combine lorsqu'on divise.

4.2 Illustration



4.3 Implémentation

Une version ni stable, ni en place

```
[ ]: import random

def tri_rapide(tab):
    if len(tab) <= 1:
        return tab
    p = random.randint(0, len(tab)-1)    # choix aléatoire du pivot
    gauche = [x for x in tab[:p]+tab[p+1:] if x < tab[p]]
    droit = [x for x in tab[:p]+tab[p+1:] if x >= tab[p]]
    return tri_rapide(gauche) + [tab[p]] + tri_rapide(droit)    # le pivot est à sa place définitive
```

4.4 Complexité

Cas	Complexité	Explication
Meilleur cas	$O(n \log n)$	Le pivot divise systématiquement le tableau en deux moitiés équilibrées.
Pire cas	$O(n^2)$	Le pivot est, à chaque appel récursif, le plus petit ou le plus grand élément; La division est extrêmement déséquilibrée, 0 élément d'un côté et $n - 1$ de l'autre.

4.5 Choix du pivot

Le choix du pivot est déterminant pour les performances réelles de l'algorithme :

- *Pivot fixe* (premier ou dernier élément) : simple à implémenter, mais catastrophique sur des données déjà triées ou presque triées, qui sont fréquentes en pratique. On tombe alors dans le pire cas.
- *Pivot aléatoire* : on choisit aléatoirement un élément pour rendre le pire cas extrêmement improbable, quelle que soit la disposition initiale des données.
- *Médiane de trois* : on prend la médiane entre le premier, le dernier et l'élément du milieu, ce qui limite statistiquement les déséquilibres tout en restant peu coûteux.